TEL200 Introduction to Robotics

Localization and Mapping

David A. Anisi

Ch. 6: Localization and Mapping

<u>Background & pre-requisites</u>

- Sailing vs. robotics
- Gaussian, White noise, PDF
- Estimation/Observer
  - Kalman Filter (KF/EKF)

<u>Localization</u>

- Dead reckoning (6.1)
- Localization with landmark/map (6.2)
- Particle Filter (PF) - Sequential Monte-Carlo Localization (6.6)

<u>Mapping</u>

- Creating a map (6.3)

<u>SLAM</u>

  - EKF SLAM (6.4)
  - Rao-Blackwellized - FastSLAM (6.7)
  - PoseGraph SLAM (6.5)

Map building – estimate the landmark positions

Given

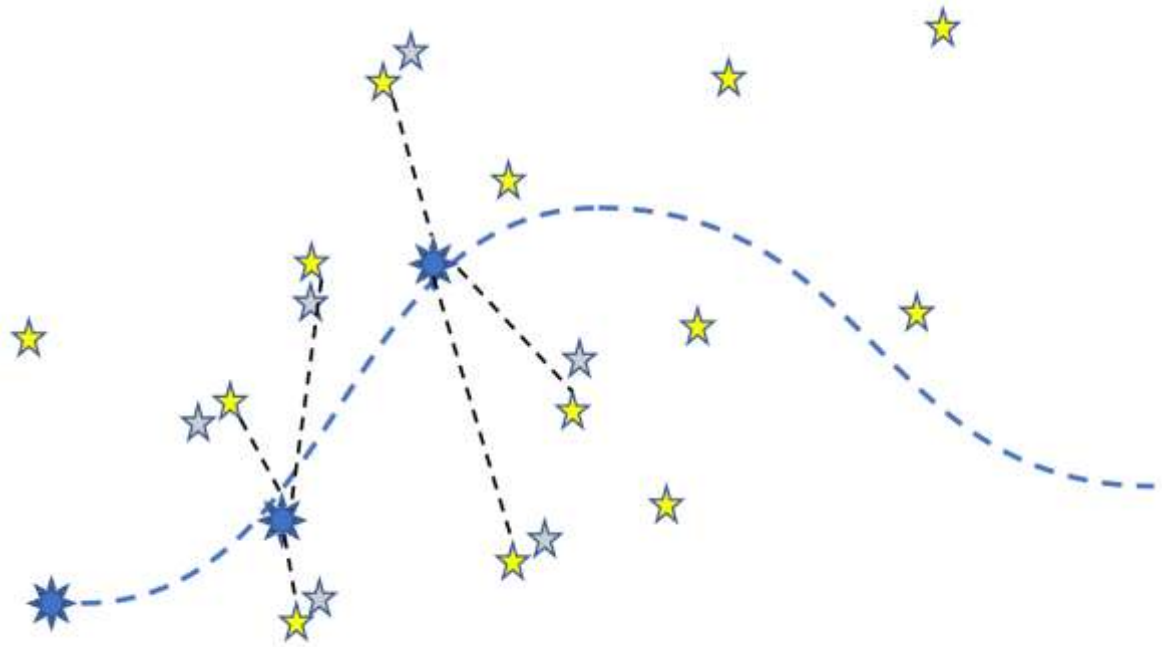
- Perfect localization, i.e., vehicle pose
- Known identity of the landmarks
- Range and bearing sensor as before

Objective

- Estimate the position of the landmarks
- Build the surrounding map

Student group activity:

1. How can this be solved using EKF?
2. What are the states to be estimated?
3. What is the size of the state vector?



Map building – estimate the landmark positions

Constructing the state vector: $\hat{\boldsymbol{x}} = \left(x_1, y_1, x_2, y_2, \cdots x_M, y_M\right)^T \in \mathbb{R}^{2M\times 1}$

Covariance Matrix: $\hat{\boldsymbol{P}} \in R^{2M\times 2M}$

Prediction equations:

$$\left.\begin{aligned} \hat{\boldsymbol{x}}^+\langle k+1\rangle &= \hat{\boldsymbol{x}}\langle k\rangle \\ \hat{\boldsymbol{P}}^+\langle k+1\rangle &= \hat{\boldsymbol{P}}\langle k\rangle \end{aligned}\right\} \text{Static, since the landmarks are stationary}$$

Coordinates of observed landmarks:

$$\boldsymbol{g}(\boldsymbol{x}, \boldsymbol{z}) = \begin{pmatrix} x_v + r\cos(\theta_v + \beta) \\ y_v + r\sin(\theta_v + \beta) \end{pmatrix}$$

State Vector Extension:

$$\boldsymbol{x}\langle k\rangle' = \boldsymbol{y}\left(\boldsymbol{x}\langle k\rangle, \boldsymbol{z}\langle k\rangle, \boldsymbol{x}_v\langle k\rangle\right) = \begin{pmatrix} \boldsymbol{x}\langle k\rangle \\ \boldsymbol{g}\left(\boldsymbol{x}_v\langle k\rangle, \boldsymbol{z}\langle k\rangle\right) \end{pmatrix}$$

Covariance Extension:

$$\hat{\boldsymbol{P}}\langle k\rangle' = \boldsymbol{Y}_z \begin{pmatrix} \hat{\boldsymbol{P}}\langle k\rangle & 0 \\ 0 & \hat{\boldsymbol{W}} \end{pmatrix} \boldsymbol{Y}_z^T \quad \text{where} \quad \boldsymbol{Y}_z = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{z}} = \begin{pmatrix} \boldsymbol{I}_{n\times n} & & \boldsymbol{0}_{n\times 2} \\ \boldsymbol{G}_x & \boldsymbol{0}_{2\times n-3} & \boldsymbol{G}_z \end{pmatrix}$$

$$\boldsymbol{G}_x = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \quad \boldsymbol{G}_z = \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{z}} = \begin{pmatrix} \cos(\theta_v + \beta) & -r\sin(\theta_v + \beta) \\ \sin(\theta_v + \beta) & r\cos(\theta_v + \beta) \end{pmatrix}$$

Estimated coordinates of the M landmarks seen. The size M=0 initially and increases every time a previously unseen landmark is observed.

Map building – estimate the landmark positions

Jacobian of $\boldsymbol{h}(\cdot)$: $$\boldsymbol{H}_{p_i} = \frac{\partial \boldsymbol{h}}{\partial \boldsymbol{p_i}} = \begin{pmatrix} \frac{x_i - x_v}{r} & \frac{y_i - y_v}{r} \\ -\frac{y_i - y_v}{r^2} & \frac{x_i - x_v}{r^2} \end{pmatrix}$$

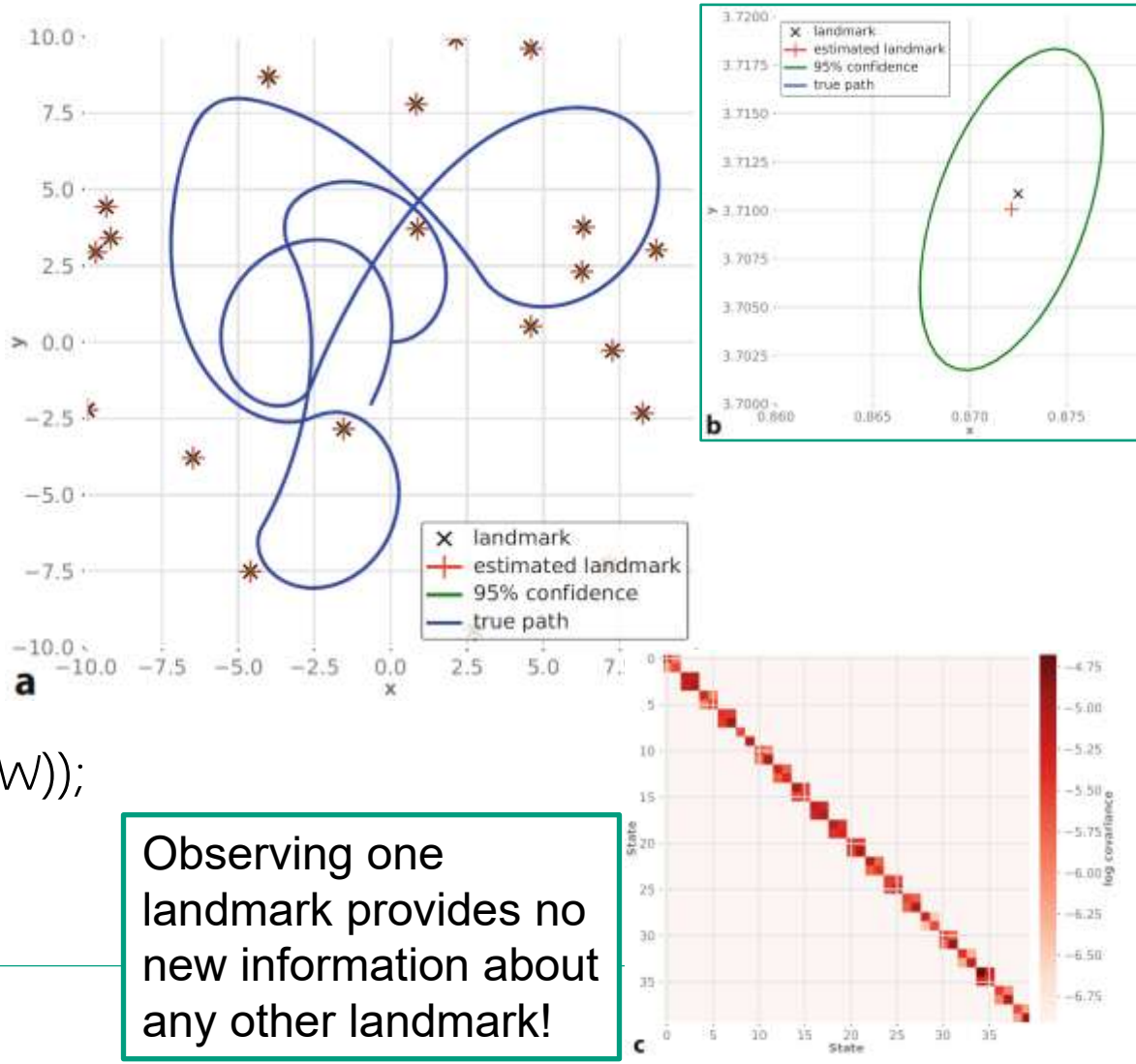
How the landmark observation changes with respect to the Landmark position for a particular robot pose.

For the mapping case: $$\boldsymbol{H}_x = \left.\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}}\right|_{\boldsymbol{w}=0} = \left(0 \cdots \boldsymbol{H}_{p_i} \cdots 0\right) \in \mathbb{R}^{2\times 2M}$$

How the landmark changes with respect to the full state vector. Now the observation depends only on the position of the landmark, Thus most entries are zeroed.

Map building – estimate the landmark positions

```
>>> map = LandmarkMap(20, workspace=10);
>>> robot = Bicycle(covar=V, animation="car");
>>> robot.control = RandomPath(workspace=map);
>>> W = np.diag([0.1, np.deg2rad(1)]) **2
>>> sensor = RangeBearingSensor(robot=robot,
map=map, covar=W, range=4, angle=[-pi/2, pi/2],
animate=True);
>>> ekf = EKF(robot=(robot, None), sensor=(sensor, W));
>>> ekf.run(T=100);
>>> map.plot(); ekf.plot_map(); robot.plot_xy();
```

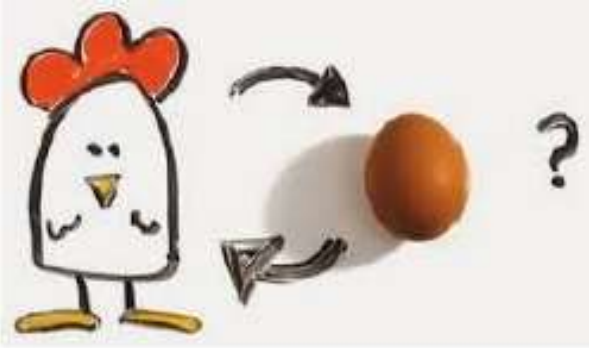


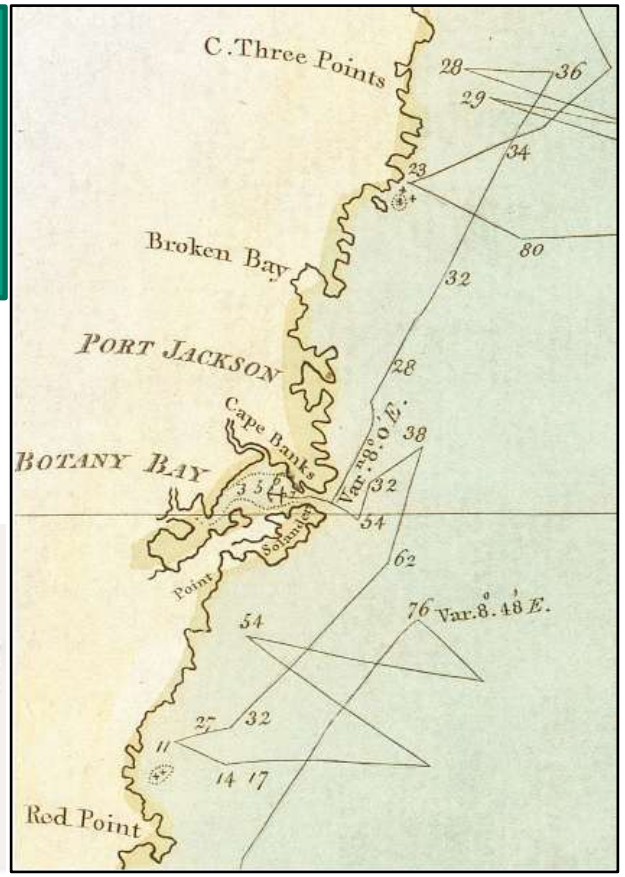
Observing one landmark provides no new information about any other landmark!

Simultaneous Localization and Mapping (SLAM)

- Localization: Map is known. State/pose need to be estimated.
- Mapping/Map-building: State/pose is assumed known, and the focus is on building the map
- SLAM: Neither position nor map known, need to estimate both at the same time
- A map is needed for localization, while a pose estimate is needed for mapping!
- Do localization and mapping in parallel!!

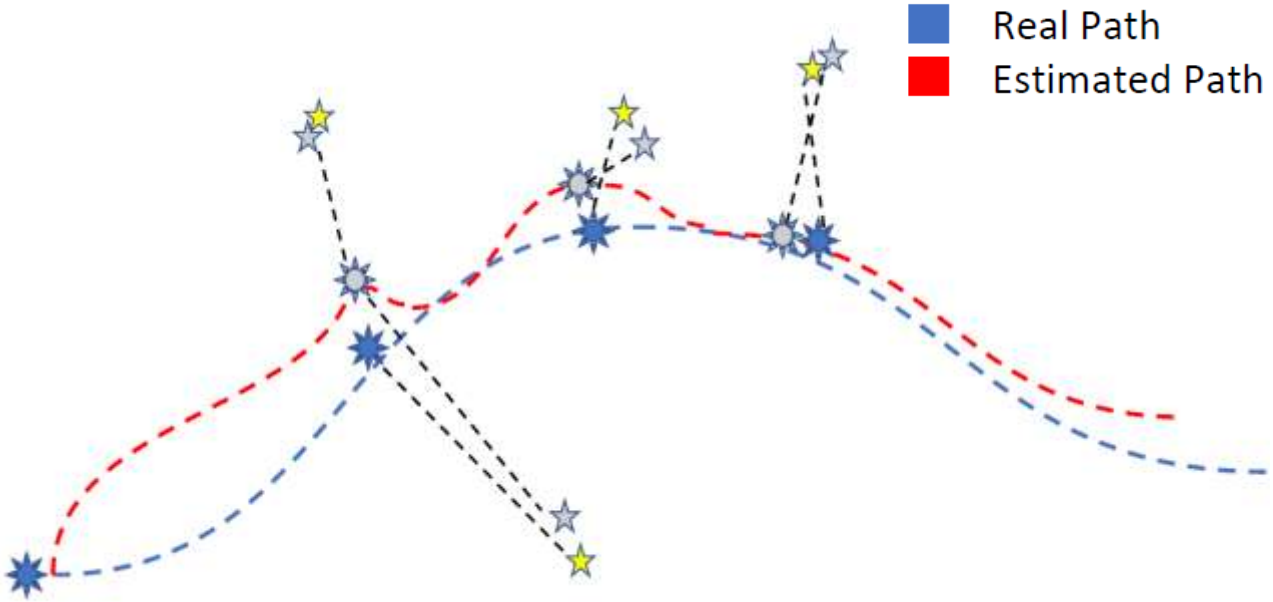
Map of the New Holland coast (now eastern Australia) by Captain James Cook in 1770. The path of the ship and the map of the coast were determined at the same time. SLAM!





Student group activity: EKF SLAM

1. How can this be solved using EKF?
2. What are the states to be estimated?
3. What is the size of the state vector?
4. How will the correlation between robot pose and landmark be?



EKF SLAM

- Constructing the state vector

$$\hat{\boldsymbol{x}} = \left(x_v, y_v, \theta_v, x_1, y_1, x_2, y_2, \cdots x_M, y_M\right)^T \in \mathbb{R}^{2M+3\times 1}$$

- Covariance matrix

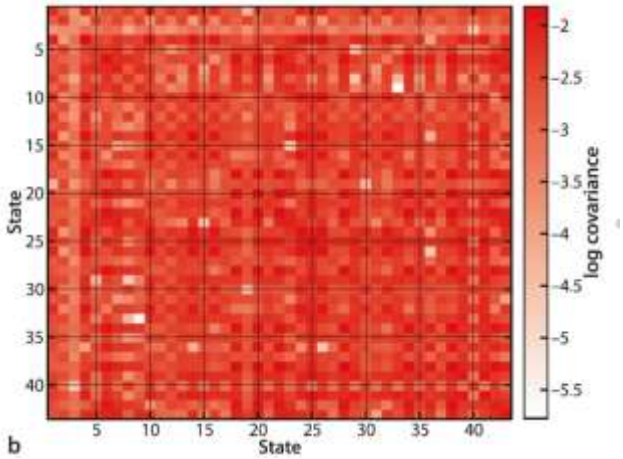
Vehicle pose

Vehicle/landmark correlation

$$\hat{\boldsymbol{P}} = \begin{pmatrix} \hat{\boldsymbol{P}}_{vv} & \hat{\boldsymbol{P}}_{vm} \\ \hat{\boldsymbol{P}}_{vm}^T & \hat{\boldsymbol{P}}_{mm} \end{pmatrix} \in \mathbb{R}^{(2M+3)\times(2M+3)}$$

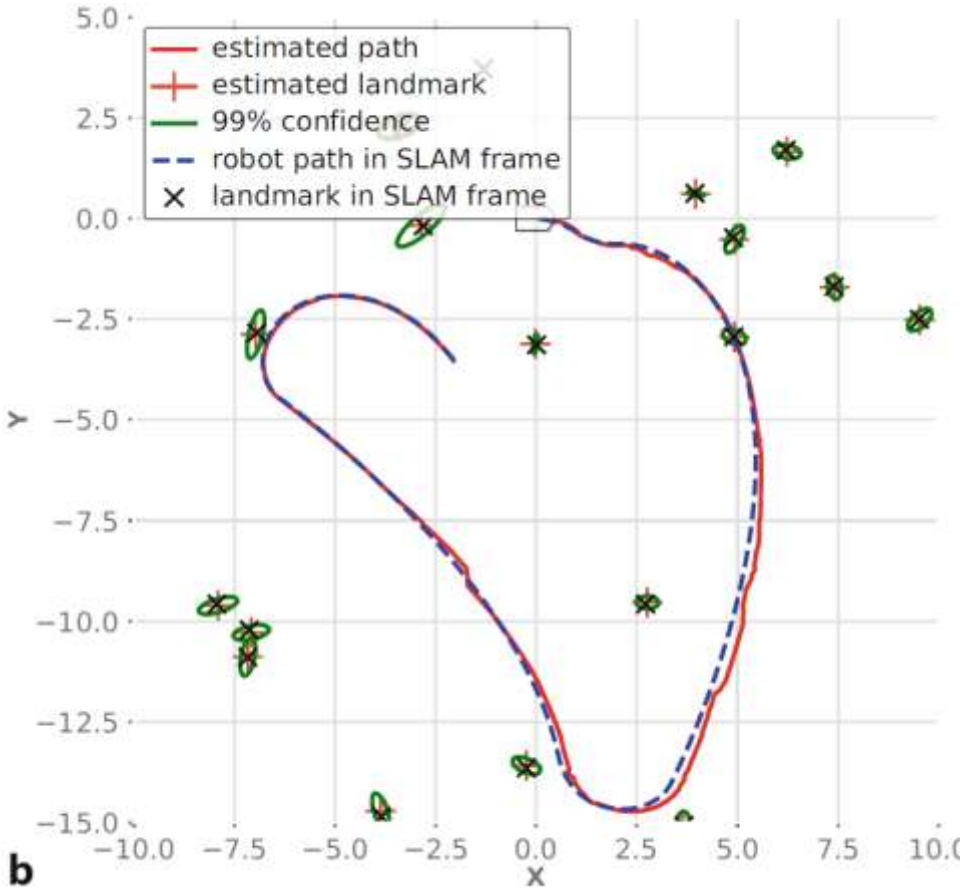
Landmark position

- The Kalman gain matrix ***K*** *distributes* innovation from the landmark observation to update *every* element of the state vector!



EKF SLAM

```
>>> map = LandmarkMap(20, workspace=10);
>>> V = np.diag([0.1, np.deg2rad(1)]) ** 2
>>> robot = Bicycle(covar=V, x0=(3, 6, np.deg2rad(-45)),
animation="car");
>>> robot.ctrl = RandomPath(workspace=map);
>>> W = np.diag([0.1, np.deg2rad(1)]) ** 2
>>> sensor = RangeBearingSensor(robot=robot, map=map,
covar=W, range=4, angle=[-pi/2, pi/2], animate=True);
>>> P0 = np.diag([0.05, 0.05, np.deg2rad(0.5)]) ** 2;
>>> ekf = EKF(robot=(robot, V), P0=P0, sensor=(sensor, W));
>>> ekf.run(T=40); map.plot(); robot.plot_xy();
>>> ekf.plot_map(); ekf.plot_ellipse(); ekf.plot_xy();
```



EKF SLAM

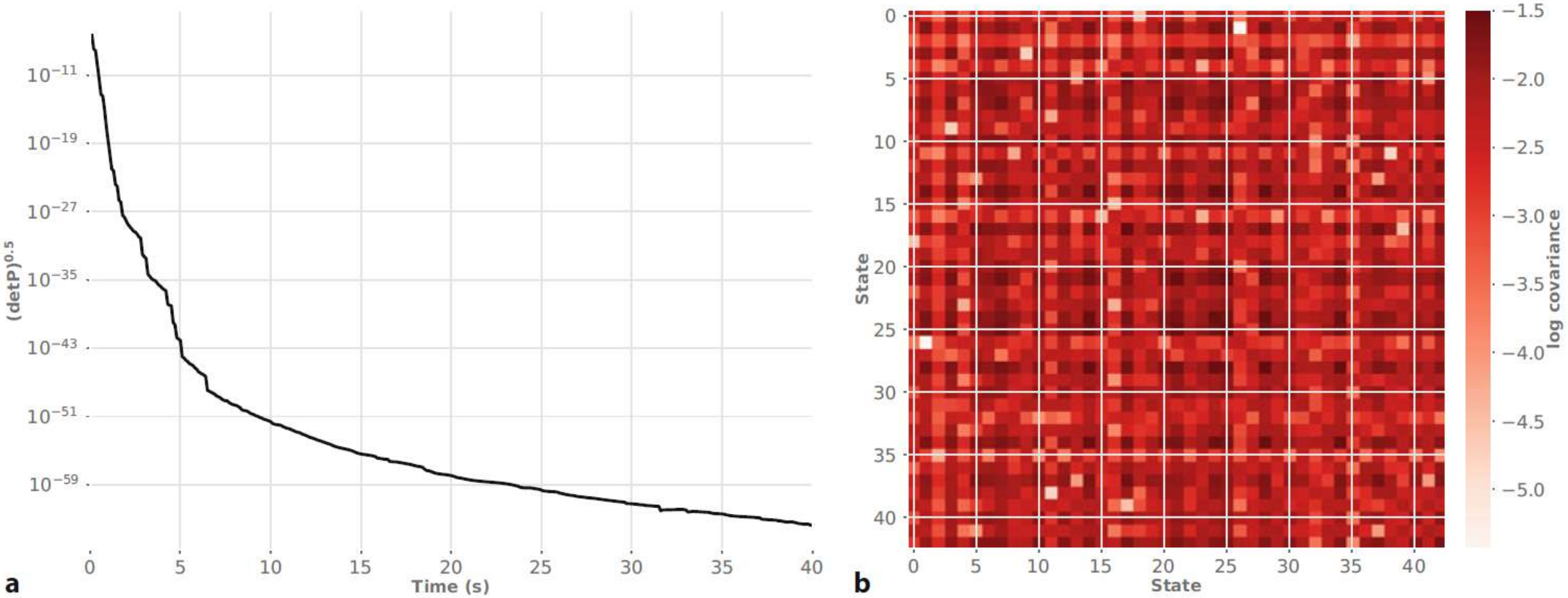


Fig. 6.14 Simultaneous localization and mapping. **a** Covariance versus time; **b** the final covariance matrix

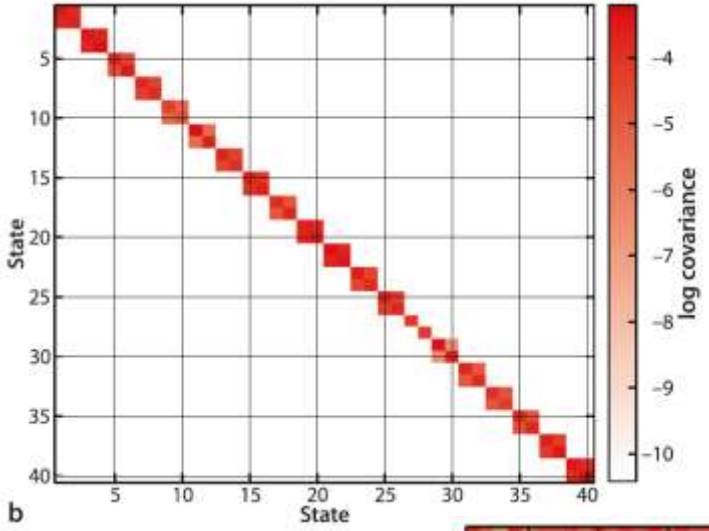
Student group activity:

1. Discuss and list some pros/cons with EKF SLAM algorithm

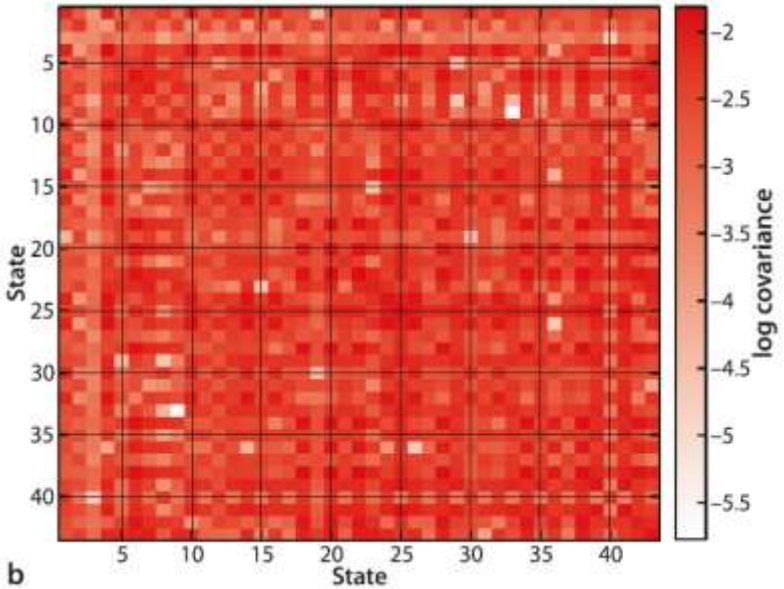
Alternative SLAM algorithm:

2. What causes the huge difference in covariance matrix structure between mapping and EKF SLAM?

3. How can we exploit this?



Cov matrix for mapping



Cov matrix for EKF SLAM

EKF SLAM

Drawbacks:

1. Size of the matrices involved increase as the size of map (#landmarks) increase
2. The covariance matrix is quadratic in #landmarks
3. Ever increasing matrix inversion cost
4. Assumption on Gaussian error distributions typically doesn't hold true
5. Ditto with linearity on system dynamics and sensor output fcn
6. Finding good estimates of covariance for process/sensor noise can be challenging
7. Cannot handle multi-modal pose estimation

Main advantage: intuitive and easy to implement

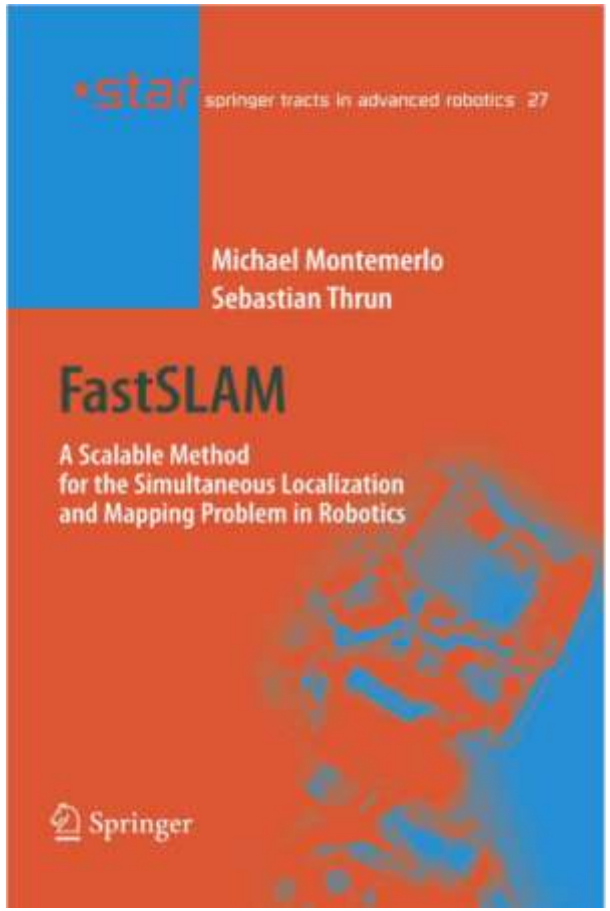
Rao-Blackwellized FastSLAM

Divide SLAM into pose estimation and map-building:

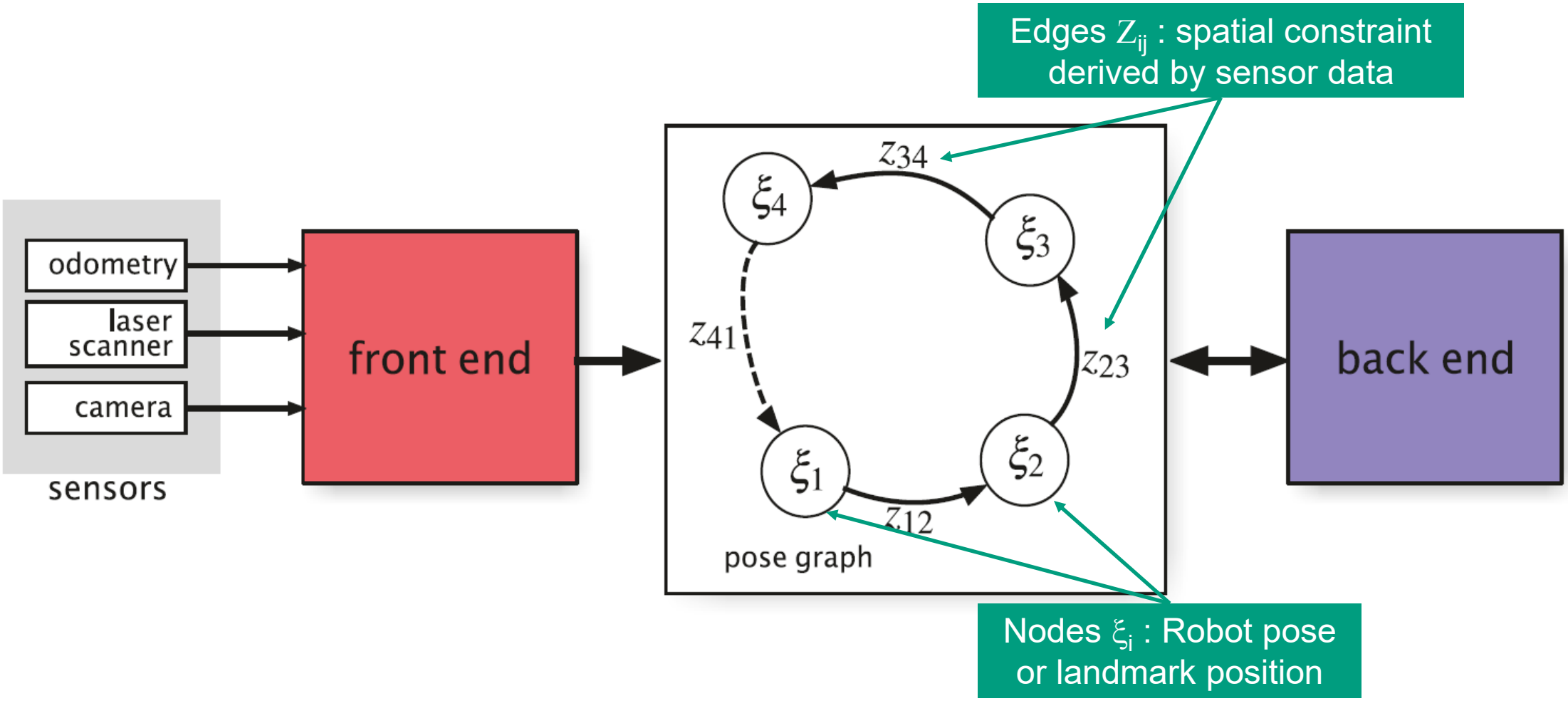
1. PF estimates pose and provides multiple hypotheses
2. EKF (or other mapping technique) handles map-building, <u>assuming</u> each pose hypothesis holds

This yields a block-diagonal covariance matrix! Instead of size 2Nx2N, we have N independent filters with size 2x2!

- O(N^2) for EKF SLAM vs. O(M log N) for FastSLAM (M = #pose hypothesis)



Graph-based SLAM



SLAM – 5min with Cyrill

Graph-based SLAM

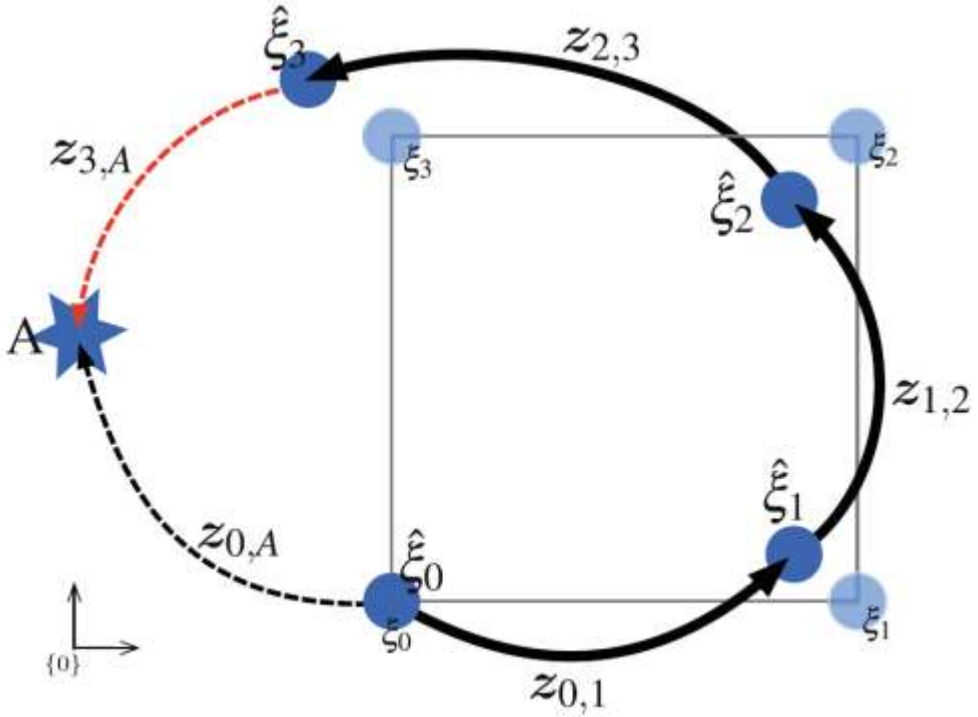
The paradigm in which most current solutions are built!

Most popular graphs:

1. PoseGraph (see Ch. 2.1.3)
2. FactorGraph

Popular back-end packages to chose from:

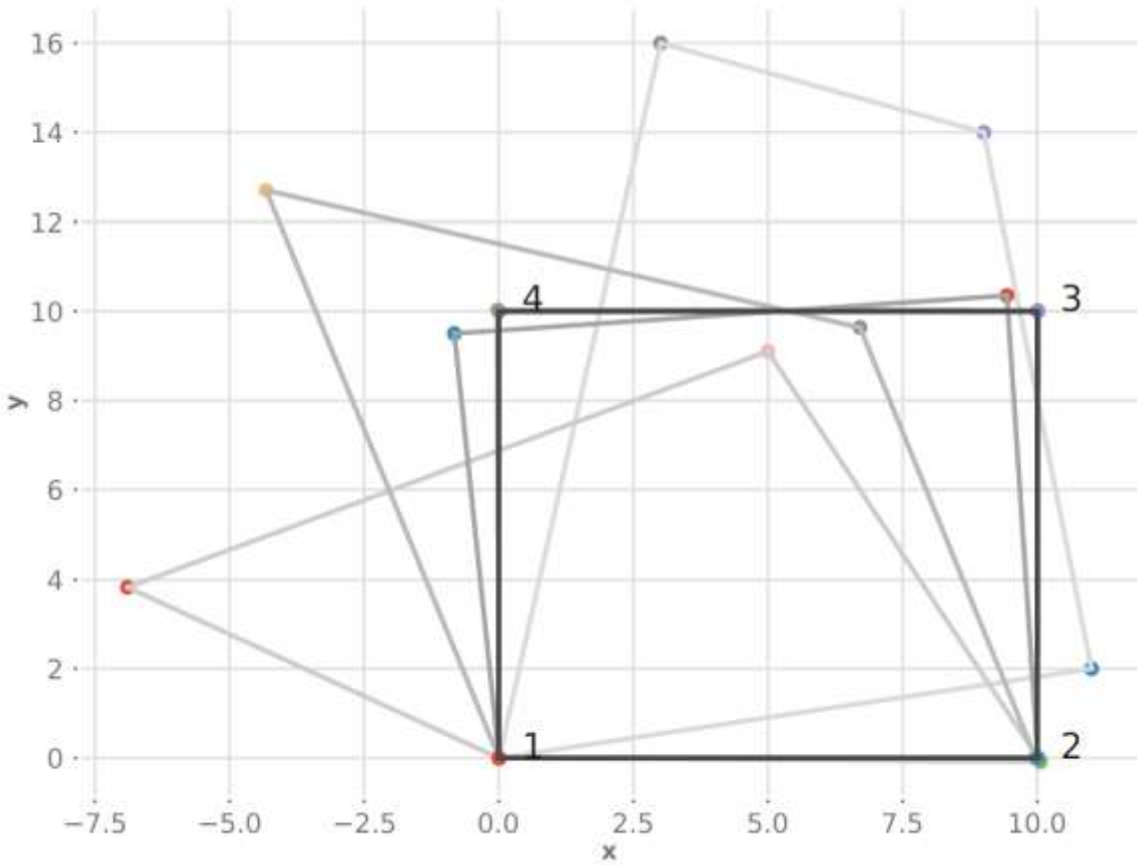
- g2o (https://github.com/RainerKuemmerle/g2o)
- gtsam (https://bitbucket.org/gtborg/gtsam)
- Google ceres (http://ceres-solver.org)



PoseGrapth SLAM Example – 4 nodes and edges

```
>>> pg = PoseGraph("data/pg1.g2o");
>>> pg.plot();
>>> pg.optimize(animate=True)
```

- File format g2o, see http://openslam.org



PoseGraph SLAM Example - 1 941 nodes and 3 995 edges

```
>>> pg = PoseGraph("data/killian-small.toro"); pg.plot(); pg.optimize()
```

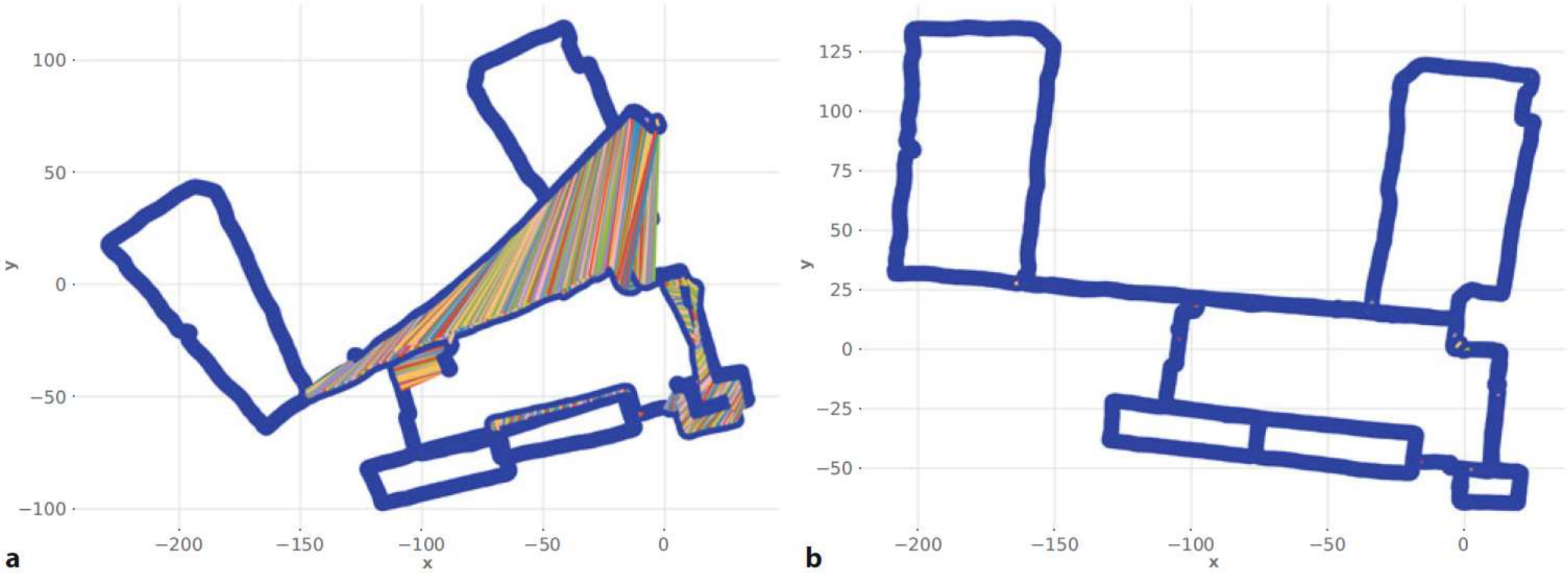


Fig. 6.18 Pose graph with 1941 vertices in blue and 3995 edges in various colors, from the MIT Killian Court dataset. **a** Initial configuration; **b** final configuration after optimization

Localization and Mapping - Recap

Localization

- Dead reckoning – prediction step in KF
  - Error accumulates, unbounded drift
- Using map with landmarks
  - KF: optimal for linear models with zero-mean Guassian noise. EKF uses the linearized model (Jacobian)
  - Uncertainty decrease by landmark observations
  - PF: supports non-linear systems and multiple hypothesis. Heavy compute

Mapping

- EKF to estimate positions of landmarks

SLAM

- EKF: to estimate robot pose and landmark position. Heavy to compute
- Rao-Blackwellized FastSLAM: PF for robot pose + EKF for landmark position
- Graph-based: PoseGraph – Large scale, sparse, least squares optimization problem

Bonus: Iterative Closest Point (ICP) – 5min with Cyrill

Thank you for your attention!